COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/07/26	**DATE SUBMITTED:**	10/09/26
NAME:	SUBALISID, JAPPETH ROYCE B. BAUTISTA, MARK JONEIL S.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values:

```
bool diff(int *x, int *y){
        for(int i = 0; i < y.length; i++){
                if(search(x, y[i])!= - 1){
                        return false;
                }
        }
        return true;
}
```

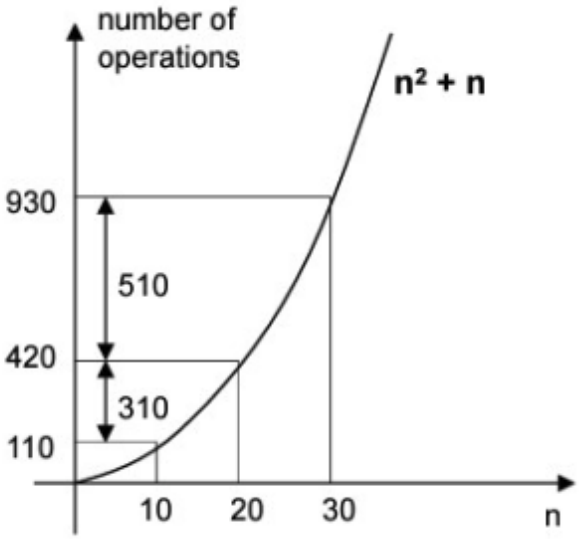
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **m * n**

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worst case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

- **Bautista** - The number of operations increases by 200 if we calculate 510 - 310 it becomes 200. If we continue to increase the size it will continue to increase by doubling or tripling the value of the numbers of operations in each array.
- **Subalisid** - If we continue to increase the size of the array by 10s, we can calculate that it goes to 1640, 2550, and 3660 operations. The increase in each step increases by 200 each time, from 0 to 110, to 310, to 510, to 710, until 1110, this shows that the growth of number of operations is linear while the increase of the number of operations is quadratic. Additionally, increasing the array size even more makes the "+ n" more insignificant therefore the worst-case time complexity becomes just O(n).

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Imagine that it is flu season and health department officials are planning to visit a school to ensure that all the enrolled children are administered their flu shot. However, there is a problem: a few children have already taken their flu shots but do not remember if they have been vaccinated against the specific category of flu that the health officials plan to vaccinate all the students against. Official records are sought out and the department is able to find a list of students that have already been administered the vaccine. A small excerpt of the list is shown here:

First Name	Last Name	Flu Shot?
1	3	Yes
2	2	No
2	3	Yes

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible. The final output of your program should look as follows:

Time taken to search = 45 microseconds
Student (836 118) needs vaccination.
Time taken to search = 37 microseconds
Student (836 118) needs vaccination.
Time taken to search = 53 microseconds
Student (836 118) needs vaccination.

The high-level steps for this implementation are as follows:

1. Represent each student as an object of the Student class.
2. Overload the required operators for the Student class so that a vector of students can be sorted using the STL std::sort() function.
3. Use a binary search to see if the student is present on the list.
4. If the student isn't present in the list, your function should return true since the student needs to be administered the vaccine.
5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.
6. Else, return false.

Step 0: Begin by including the following headers:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
```

Step 1: Define the STUDENT class as follows:

```
class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;
    public:
        //constructor
        Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){}

        // Getters
        auto get_name() { return name;}
        auto is_vaccinated() { return vaccinated; }
```

Step 2: Overload the required operators for the Student class so that a vector of students can be sorted using the STL std::sort() function.

```
// Two people are same if they have the same name
        bool operator ==(const Student& p) const {
        return this->name == p.name;
        }
        // The ordering of a set of people is defined by their name
        bool operator< (const Student& p) const {
        return this->name < p.name;
        }
        bool operator> (const Student& p) const {
        return this->name > p.name;
```

Step 3: The following function lets us generate a student from random data:

```
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type>
uniform_dist(1, max);
    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand),
uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}
```

For the next important function, we will implement the use of **std::chrono for experimental analysis.**
The following code is used to run and test the output of our implementation.

```
void search_test(int size, Student p) {
    std::vector<Student> people;
    // Create a list of random people
    for (auto i = 0; i < size; i++)
    people.push_back(generate_random_Student(size));
    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
    std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);
```

```
    // Stop the clock
    std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
    std::chrono::duration_cast<std::chrono::microseconds> (end -
begin).count() <<
    " microseconds" << std::endl;

    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
        << ") " << "needs vaccination." << std::endl;
    }else {
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
        << ") " << "does not need vaccination." << std::endl;
    }

}
```

5. The following function implements our logic for whether a vaccination is needed:

```
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false) {
            return true;
        } else if (mid_element == P && mid_element.is_vaccinated() ==
true){
            return false;
        } else if (mid_element > P){
            std::advance(last, -mid_element_index);
```

```
        }

        if (mid_element < P){
            std::advance(first, mid_element_index);
        }
        // Student not found in the sequence and therefore should be
vaccinated
        if (range_length == 1){
            return true;
        }
    }
}
```

6. Finally the driver code is implemented as follows:

```
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Fill out the table below and include in section 6 table 11-1.

- **Bautista**

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	0us	Time taken to search (1000 elements) = 0 microseconds Student (877 481) needs vaccination.	Searching the smallest input size will result in the shortest amount of time used.
10,000	0us	Time taken to search (10000 elements) = 0 microseconds Student (877 481) needs vaccination.	Searching difference between 1,000 and 10,000 will result by half a second
100,000	1us	Time taken to search (100000 elements) = 1 microseconds Student (877 481) needs vaccination.	The searching difference of 100,000 will output the longest than the two.

- **Subalisid**

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	1us	Time taken to search = 1 microseconds Student (549 129) needs vaccination.	Searching for a target in a small sample size will result in the shortest time.
10,000	1us	Time taken to search = 1 microseconds Student (549 129) needs vaccination.	The difference in search time between 1000 and 10000 is negligible in terms of microseconds.
100,000	3us	Time taken to search = 3 microseconds Student (549 129) needs vaccination.	There is a noticeable difference in search time when the sample size was increased 10 more times, resulting in around 3x longer time.

Provide your own analysis of the outcome listed in this table.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis:

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
        for i = 0 to n-1 do
                for j = i+1 to n-1 do
                        if A[i] equals to A[j] then
                                return false
    return true
```

Analyses:

- **Bautista**
  - **Theoretical Analysis -** The algorithm checks every element of the array against the other elements to determine if there area duplicate values. Since it uses two nested loops, its time complexity is O(n^2). Meaning that the number of comparisons increases significantly as the size of the array increases.

- **Experimental Analysis -** Based on the experimental results the runtime increases as the output increases as the input size becomes larger, although the measured time may fluctuate slightly between different runs. The small variations can be caused by the computer's processor, background processes, compiler optimization and other system conditions.

- **Analysis and Comparison -** The experimental results generally support the theoretical analysis because the algorithm requires more comparisons as the input size increases. However, the measured runtime may not increase perfectly because the actual execution time is affected by the computer system and the very small amount of time being measured.

- **Subalisid**
  - **Theoretical Analysis** - the outer for-loop runs for n times, while the inner for-loop also runs for n times. The entire algorithm then must run with a time complexity of $O(n^2)$.
  - **Experimental Analysis** -

```
Time taken to search = 0 microseconds
All elements are unique
```
- 10 numbers

```
Time taken to search = 3 microseconds
All elements are unique
```
- 50 numbers

```
Time taken to search = 18 microseconds
All elements are unique
```
- 100 numbers

    - We can see from the screenshots that increasing the number of elements in the array will increase the runtime significantly since the time complexity squares the number of elements. With 10 elements, the time is almost 0us, while with 50 it increased to about 3us, and in 100 elements, it increased by 6x compared to 50 elements.

  - **Analysis and Comparison** - The theoretical analysis says that the time complexity must grow accordingly with the number of input elements. In theory, with 50 elements, there should be 2,500 comparisons and with 100, there should be 10,000, meaning a 4x increase in runtime. In the experiment, we saw a 6x increase in runtime from 3us to 18 us, which should be negligible since it could be affected by other factors such as machine specs and randomness. This shows that the theoretical analysis is correct.

Problem 2: Consider another program that raises a number x to an arbitrary non-negative integer, n.

```
Algorithm rpower(int x, int n):
        1 if n == 0 return 1
        2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
        1 if n == 0 return 1
        2 if n is odd then
        3 y = brpower(x, (n-1)/2)
        4 return x*y*y
        5 if n is even then
        6 y = brpower(x, n/2)
        7 return y*y
```

Analysis:

- **Bautista**
  - **Theoretical Analysis -** The first algorithm calculates the power by recursively reducing the exponent by one, giving it a time complexity of O(n). The second algorithm uses the divide and conquer approach by repeatedly dividing the exponent by two giving it a much more efficient time complexity of O(log n)

  - **Experimental Analysis -** The experimental results were measured in nanoseconds (ns). The runtimes may vary between different computer systems because of the differences in processor performance, compiler optimization and background processes and system load. The results show that the second algorithm generally requires less time than the first algorithm as the input size becomes larger.

  - **Analysis and Conclusion -** The experimental results are consistent with the theoretical analysis because the divide and conquer power algorithm is more efficient than the first algorithm for larger inputs. The first algorithm performs more recursive operations as the exponent increases, while the second algorithm reduces the problem size by half at each step. Therefore O(log n) is significantly more efficient than O(n) for large values of n.

- **Subalisid**
  - **Theoretical Analysis -** The first algorithm runs and reduces the power by 1 every recursive call, therefore, it should run n times—n being the exponent. The second algorithm divides the exponent in half every iteration, therefore, it should have a time complexity of log n. Combining the two algorithms, we should get a time complexity of O(n + log n), but since log n is smaller than n when its size increases, the worst-case time complexity should simplify to O(n)

  - **Experimental Analysis -**

    ```
    Time taken to run = 0 microseconds
    ```
    n = 10

    ```
    Time taken to run = 1 microseconds
    ```
    n=100

    ```
    Time taken to run = 30 microseconds
    ```
    n=1000

    ```
    Time taken to run = 357 microseconds
    ```
    n=10000

    - If we look at the results of the experiment, there seems to be a non linear growth from 100 to 1000, but since there is such a small amount of difference, we can take it as a negligible time difference. In 1000 to 10000 however, we can see an almost 10x increase in runtime.
    -

  - **Analysis and Conclusion -** Comparing my theoretical analysis with the experimental analysis, I can see that they are somewhat consistent. Since we are taking a time complexity of O(n + log n), there will be a little bit of deviation from the linear growth of runtime because of the additional log n time. We later saw in the latter part of the experiment that increasing the n size will show the time complexity clearer and clearer since the log n becomes less and less significant to the time. Therefore my theoretical analysis can be assumed to be correct.

3. CONCLUSION

This hands-on activity helped us understand better how to analyze algorithms and determine their time complexities. It demonstrated the change in different time complexities as n increases. We also learned the difference in theoretical and experimental analysis which is useful in determining if the algorithm will run as expected or not. Doing the activity on separate computers allowed us to see the effect of machine specs on the runtime of a program which is the main concern of determining time complexity. In the end, this activity was a great practice for us to better understand time complexity and its application.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.